CSC 212: Data Structures and Abstractions

09: Binary Heaps, BuildHeap

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Spring 2026

THINK BIG WE DO℠

Designing a max-PQ using a binary max-heap

```
class PriorityQueue {
    private:
        int *arr;
        size_t capacity_;
        size_t size_;

        size_t parent(size_t i) { return (i-1) / 2; }
        size_t left(size_t i) { return 2*i + 1; }
        size_t right(size_t i) { return 2*i + 2; }

        void upHeap(size_t i);
        void downHeap(size_t i);

    public:
        PriorityQueue(size_t cap) : capacity_(cap), size_(0) {
            arr = new int[capacity_];
        }
        ~PriorityQueue() {
            delete [] arr;
        }

        void enqueue(const int& val);
        void dequeue();

        const int& top() const { return arr[0]; }
        size_t size() { return size_; }
        size_t capacity() { return capacity_; }
        bool empty() { return size_ == 0; }
};
```

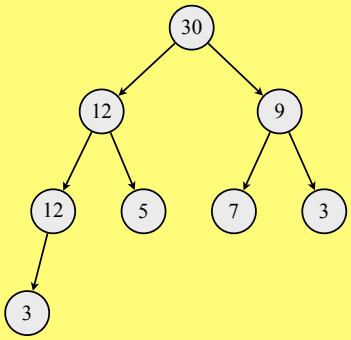

Enqueue (**push**)

- Algorithm (min-heap is analogous)
  1. append element to the end of the array
  2. "bubble up" (a.k.a. `upHeap`)
     - **set** `idx` to the newly added element's position
     - **while** `idx` is not the root **and** element at `idx` is greater than its parent
       - swap them and update `idx` to the parent's position
- Time complexity
  - ✓ how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap

Practice

- Enqueue 20
  - ✓ show resulting array
- Enqueue 1
  - ✓ show resulting array
- Enqueue 50
  - ✓ show resulting array



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
30	12	9	12	5	7	3	3								

size

enqueue and upHeap

```
void PriorityQueue::enqueue(const int& val) {
    if (size_ == capacity_) {
        throw std::out_of_range("PriorityQueue is full");
    }
    arr[size_] = val;
    size_ ++;
    upHeap(size_-1);
}

void PriorityQueue::upHeap(size_t idx) {
    while (idx > 0) {
        size_t p = parent(idx);
        if (arr[idx] > arr[p]) {
            std::swap(arr[idx], arr[p]);
            idx = p;
        } else {
            break;
        }
    }
}
```

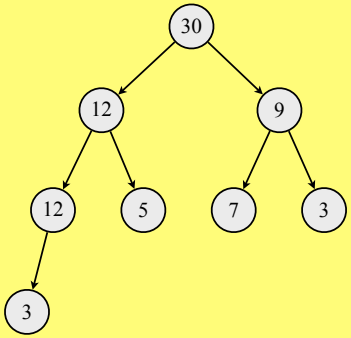

Dequeue (**pop**)

- Algorithm (min-heap is analogous)
  1. swap the root with the last element in the array
  2. remove the last element from the array (return this value if needed)
  3. "bubble down" (a.k.a. `downHeap`)
     - **set** `idx` to the root element's position
     - **while** `idx` is not a leaf **and** element at `idx` is smaller than its largest child
       - swap them and update `idx` to that child's position
- Time complexity
  - ✓ how many swaps are necessary in the worst case? $\Theta(\log n)$

https://visualgo.net/en/heap

Practice

- Dequeue
  - ✓ show resulting array
- Dequeue
  - ✓ show resulting array
- Dequeue
  - ✓ show resulting array



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
30	12	9	12	5	7	3	3								

size

dequeue and downHeap

```
void PriorityQueue::dequeue() {
    if (size_ == 0) {
        throw std::out_of_range("PriorityQueue is empty");
    }
    arr[0] = arr[size_-1];
    size_ --;
    downHeap(0);
}

void PriorityQueue::downHeap(size_t i) {
    while (true) {
        size_t largest = i;
        size_t l = left(i);
        size_t r = right(i);
        if (l < size_ && arr[l] > arr[largest]) {
            largest = l;
        }
        if (r < size_ && arr[r] > arr[largest]) {
            largest = r;
        }
        if (largest != i) {
            std::swap(arr[i], arr[largest]);
            i = largest; // move down to largest
        } else {
            break;
        }
    }
}
```



```
#include <iostream>
#include <stdexcept>

int main() {
    PriorityQueue pq(100);

    pq.enqueue(3);
    pq.enqueue(1);
    pq.enqueue(5);
    pq.dequeue();
    pq.enqueue(2);
    pq.dequeue();
    pq.enqueue(7);
    pq.dequeue();
    pq.dequeue();
    pq.enqueue(7);
    pq.enqueue(7);

    while (! pq.empty()) {
        std::cout << pq.top() << std::endl;
        pq.dequeue();
    }

    return 0;
}
```


Performance (priority queues)

Method	Unsorted Array	Sorted Array	Binary Heap
Enqueue	O(1)	O(n)	O(log n)
Dequeue	O(n)	O(1)	O(log n)
Top	O(n)	O(1)	O(1)
Size	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)

Practice

- Given a stream of integers, design an algorithm to efficiently report the **k-th largest element** seen so far at any time.
  - ✓ as new numbers arrive, your algorithm must update its data structure and return the current k-th largest number
- Sample input:
  - ✓ k = 3
  - ✓ values = {4, 5, 8, 2, 3, 5, 10, 9, 4}
- Output:
  - ✓ - - 4 4 4 5 5 8 8
  - ✓ k-th largest element after every insertion

Results Table — Min-Heap of Size k = 3

Step	val	Action	Heap	k-th Largest
0	4	push	{4}	—
1	5	push	{4, 5}	—
2	8	push	{4, 5, 8}	**4**
3	2	discard	{4, 5, 8}	**4**
4	3	discard	{4, 5, 8}	**4**
5	5	pop 4 → push 5	{5, 8, 5}	**5**
6	10	pop 5 → push 10	{5, 8, 10}	**5**
7	9	pop 5 → push 9	{8, 10, 9}	**8**
8	4	discard	{8, 10, 9}	**8**

Practice

- Given an array of integers and a window size k, design an algorithm that returns a new array where each element represents the **maximum value** within a sliding window of size k, as it moves from left to right across the input array
- Sample input:
  - ✓ k = 4
  - ✓ values = {1, 13, -1, -3, 5, 3, 6, 7}
- Output:
  - ✓ {13, 13, 5, 6, 7}

This sliding-window maximum is directly analogous to **max pooling** in neural networks

Sliding Window Maximum — Lazy Deletion Max-Heap

k = 4, values = {1, 13, −1, −3, 5, 3, 6, 7}

Heap stores **(val, idx)** pairs in max-heap array order. Expiry threshold at step i: index j is expired when j ⩽ i − k.

i	val	Action	Heap	Window Max
0	1	push	{(1,0)}	—
1	13	push	{(13,1),(1,0)}	—
2	−1	push	{(13,1),(1,0),(−1,2)}	—
3	−3	push; top (13,1) valid	{(13,1),(1,0),(−1,2),(−3,3)}	**13**
4	5	push; top (13,1) valid	{(13,1),(5,4),(−1,2),(−3,3),(1,0)}	**13**
5	3	push; pop expired (13,1)	{(5,4),(1,0),(3,5),(−3,3),(−1,2)}	**5**
6	6	push; top (6,6) valid	{(6,6),(1,0),(5,4),(−3,3),(−1,2),(3,5)}	**6**
7	7	push; top (7,7) valid	{(7,7),(1,0),(6,6),(−3,3),(−1,2),(3,5),(5,4)}	**7**

Practice

- How to build a max-heap **from an existing array** (vector)?
  - ✓ show the algorithm
  - ✓ analyze the computational cost

Bulding a max-heap in linear time

buildHeap (in linear time)

- Given
  - ✓ an **unsorted array A of n elements**

(some elements may violate the heap property)

- Algorithm
  1. **set** `idx` to the last non-leaf node's position: `parent(n-1)`
  2. **while** `true`
     - perform `downHeap` on node `idx`
     - if `idx == 0` then break
     - decrement `idx`

Why $\Theta(n)$? Most nodes are near the bottom and have small subtree height. Each downHeap costs $\Theta(h)$. Summing over all heights yields $\Theta(n)$.

https://visualgo.net/en/heap

Practice

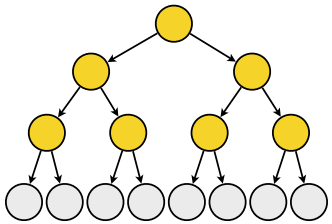
- Build a max-heap from the following array using buildHeap
  - ✓ 10 42 25 13 17 33 45 50

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

```
set idx to parent of last node
while idx >= 0
   perform downHeap on node idx
   decrement idx
```


Analysis

- Total cost
  - ✓ sum of the costs of calling **downHeap** for all internal nodes (sum of heights)



lets assume a completely filled binary tree of height h

$$T(n) = 1(h) + 2(h-1) + 4(h-2) + \ldots + 2^h(0)$$

$$= \sum_{i=0}^{h} 2^i(h-i)$$

$$= h\sum_{i=0}^{h} 2^i - \sum_{i=0}^{h} i2^i = \ldots \quad \ldots \quad \ldots = \Theta(n)$$

Performance (priority queues)

Method	Unsorted Array	Sorted Array	Binary Heap
Enqueue	O(1)	O(n)	O(log n)
Dequeue	O(n)	O(1)	O(log n)
Max/Min	O(n)	O(1)	O(1)
Size	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)
Enqueue N	O(n)	$O(n^2)$	O(n)**

(**) assuming we use **buildHeap**

Practice

- What is the output of this code?
  - ✓ also indicate the cost of each line

```
void printHeap(std::priority_queue<int> heap) {
    while (!heap.empty()) {
        std::cout << heap.top() << " ";
        heap.pop();
    }
    std::cout << std::endl;
}

int main() {
    std::priority_queue<int> heap1;  // max-heap by default
    heap1.push(3);
    heap1.push(1);
    heap1.push(4);
    heap1.push(2);
    std::cout << heap1.top() << std::endl;

    std::vector<int> data = {-3, -1, -4, -2};
    std::priority_queue<int> heap2(data.begin(), data.end());
    std::cout << heap2.top() << std::endl;

    printHeap(heap1);
    printHeap(heap2);

    return 0;
}
```

what happens if we pass a reference instead?

Practice

- Given an array of points in 2D space and an integer k, design an algorithm to return the k points closest to the origin (0,0) — using the Euclidean distance
- Sample input:
  - ✓ k = 2
  - ✓ points = {{1,3}, {-2,2}, {5,8}, {0,1}}
- Output:
  - ✓ {{0,1}, {-2,2}}